GOMEZ Y CRESPO

COSTE MP · APLICACIÓN INTERNA

Documentación técnica

Para quien mantiene y modifica la aplicación

Gómez y Crespo · Generado el 28/10/2026 a partir de `docs/DOCUMENTACION_TECNICA.md` del repositorio

Índice

1. Qué hace
2. Arquitectura
   Stack
3. Estructura del repositorio
4. Entorno de desarrollo
5. El cálculo (desglose.py)
   5.1 La consulta
   5.2 Datos del artículo raíz
   5.3 Árbol, rollup y avisos
   5.4 Caché y coste/hora
6. Publicación (exportar_costes.py)
   Objetos de Postgres (esquema core)
7. Web
8. Tests
   Gates
9. Despliegue
10. Recetas
11. Trampas conocidas y deuda técnica
12. Checklist de relevo

Documento de relevo para quien vaya a mantener la aplicación. Explica la arquitectura, el flujo de datos, el cálculo, la publicación en Postgres, el despliegue y las trampas conocidas. Para incidencias del día a día ver GUIA_SOPORTE.pdf.

Otros documentos del repositorio:

Archivo	Contenido
`reglas-del-coste.html`	**Referencia de negocio**: todas las condiciones del cálculo (qué se despieza, de dónde sale el precio y el tiempo, cada aviso), con los casos reales que provocaron cada regla. Léela antes de tocar `desglose.py`
`despliegue.md`	Preparación del servidor: montaje CIFS de la carpeta compartida, `.env`, comprobaciones
`README.md`	Uso rápido, Docker, tests y gate de `pre-push`
Comentarios de `sql/*.sql` y docstrings	Explican el porqué de cada tabla, vista y decisión. Son fiables y están al día

> **Estado a 28/10/2026.** `main` (= `origin/main`, commit `77e8bc9`) no incluye todavía el trabajo en curso de la copia de desarrollo: página `/configuracion` (`web_config.py`, `sql/06`), coste/hora en Postgres, detalle por pieza e histórico (`sql/07`), páginas `/evolucion` y `/piezas` (`web_analisis.py`), lote en segundo plano y sus tests. Este documento describe la aplicación **con** ese trabajo. Hasta que se suba y despliegue, producción no tiene esas partes.

1. Qué hace

Calcula el **coste de fabricación** de cualquier artículo del ERP `GOMEZYCRESPO`:

1. Explota el escandallo a todos los niveles con un CTE recursivo en SQL Server.
2. Valora cada pieza comprada (último albarán → tarifa de proveedor → subcontrata).
3. Calcula el coste de operación (minutos × coste/hora de operario).
4. Señala lo que falta (avisos) y quién debe arreglarlo.

Y lo hace en tres modos:

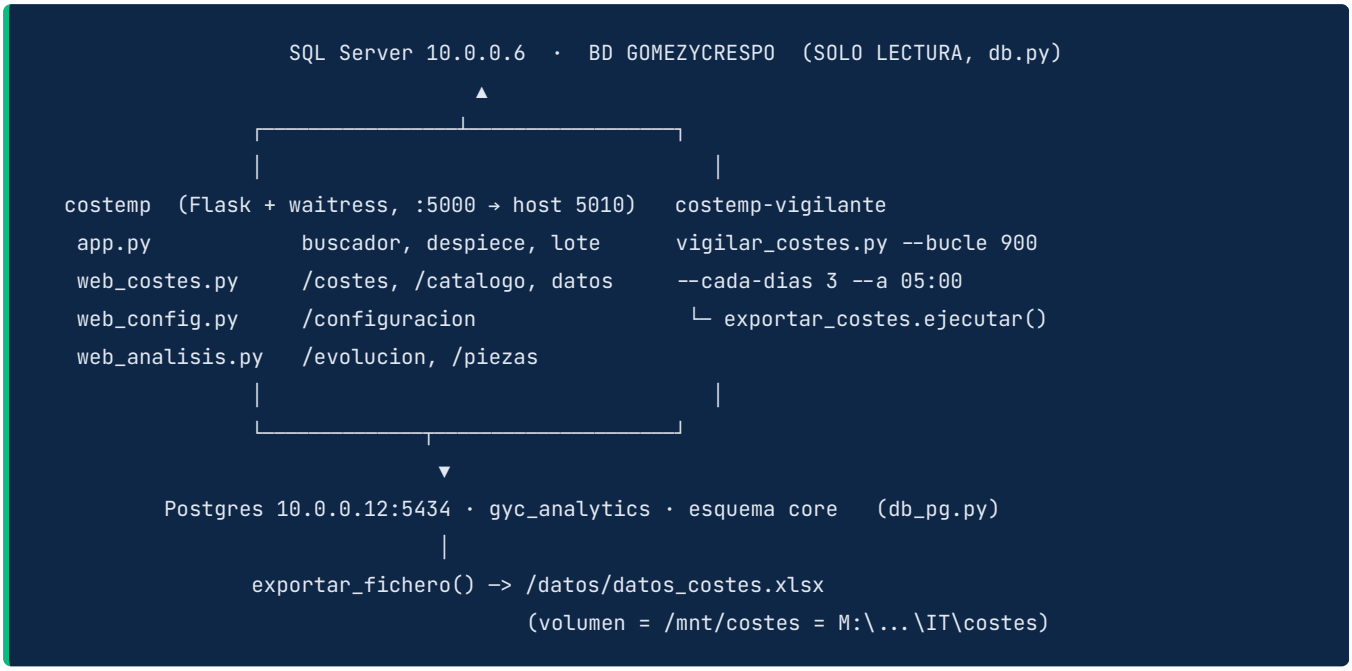
Modo	Dónde	Fuente
Un artículo al momento	Buscador `/`	ERP (caché de 10 min)
Un Excel de artículos	`/lote`	Postgres para los del catálogo, ERP para el resto
El catálogo entero (~550)	Pasada cada 3 días o `/costes` → Recalcular	ERP → se publica en Postgres

Lo publicado en Postgres alimenta `/catalogo`, `/costes`, `/evolucion`, `/piezas`, el libro `datos_costes.xlsx` (Power Query del libro de análisis de Finanzas) y Power BI.

Principio de diseño que recorre todo el código: *una sola verdad*. El cálculo vive en `desglose.py` y nada más lo reimplementa: la pasada, el lote y la web llaman a las mismas funciones (`desglose`, `construir_arbol`, `avisos_arbol`). Las tablas y vistas de Postgres son una foto, sin lógica de cálculo. Si añades un paso,

añádelo en el sitio compartido.

2. Arquitectura



- Misma imagen Docker para los dos contenedores; solo cambia el `command`.
- Cada proceso tiene su propia caché en memoria (`desglose._cache`, `_coste_hora`).
- No hay usuarios ni autenticación. `/configuracion` puede pedir `CONFIG_CLAVE`.

Stack

Python 3.12 · Flask · waitress · pandas · openpyxl · SQLAlchemy · pyodbc (ODBC Driver 18 for SQL Server) · psycopg2 · python-dotenv · pytest. Frontend: plantillas Jinja con HTML, CSS y JS sin framework ni build.

3. Estructura del repositorio

Archivo	Qué hace
`desglose.py`	**Núcleo.** SQL del CTE recursivo (precio, valorable, duplicados, `NO_SUMA_PROPIA`), búsqueda, tiempo de operación, caché, `construir_arbol` (rollup), `avisos_arbol`, `lineas_coste`, export Excel de un artículo y CLI (`py desglose.py <codigo>`)
`app.py`	Flask: `/`, `/api/buscar`, `/api/desglose`, `/api/recalcular`, `/api/escandallo`, `/api/excel`, `/lote*`. Registra los blueprints
`web_costes.py`	`/costes` (estado + recalcular en hilo), `/costes/pendientes`, `/catalogo`, `/catalogo/datos`, `/catalogo/<id>/huecos`, `/costes/datos.xlsx`
`web_config.py`	`/configuracion`: parámetros declarados en `PARAMETROS`, guardado con historial
`web_analisis.py`	`/evolucion` y `/piezas` (usos de una pieza y simulador de precio). Solo Postgres

Archivo	Qué hace
`exportar_costes.py`	La pasada: calcula el catálogo y publica de forma atómica en Postgres + escribe el Excel. CLI con `--crear`, `--solo-pendientes`, `--limite`, `--origen`
`vigilar_costes.py`	Bucle del contenedor vigilante: decide si toca pasada (cada N días desde la hora H)
`validar_costes.py`	Red de seguridad: compara lo publicado con lo que calcula la app ahora (`--todos`)
`db.py` / `db_pg.py`	Engines SQLAlchemy cacheados al ERP y a `gyc_analytics`
`sql/01…07_*.sql`	Tablas y vistas de `core` (idempotentes, se aplican en orden)
`templates/*.html`	Una plantilla por página
`tests/`	Tests (ver apartado 8)
`docs/`	Esta documentación, reglas del coste y despliegue
`scripts/git-hooks/pre-push`	Gate local: no deja hacer push si fallan los tests
`.github/workflows/tests.yml`	CI: los mismos tests en el runner self-hosted
`scratch_*.py`	Scripts de usar y tirar (ignorados por git)

4. Entorno de desarrollo

Requisitos: Python 3.12 con el launcher `py`, **ODBC Driver 18 for SQL Server**, acceso de red a `10.0.0.6` (ERP) y `10.0.0.12:5434` (Postgres). Docker Desktop para el gate de tests.

```
py -m pip install -r requirements-dev.txt
copy .env.example .env          # rellenar credenciales (pedir a sistemas)
py app.py                       # http://127.0.0.1:5000  (modo debug de Flask)
py desglose.py 12101021         # despiece por línea de comandos → salidas/
```

Variables del `.env`:

Variable	Para qué
`SQLSERVER_HOST/DB/USER/PASS`	ERP, usuario **de solo lectura**
`PG_HOST/DB/USER/PASS/PORT`	`gyc_analytics` (puerto 5434 en el servidor)
`COSTES_SALIDA_XLSX`	Dónde escribir `datos_costes.xlsx`. En Docker lo fija el compose a `/datos/...`
`COSTES_DATOS_DIR`	Carpeta del host que se monta en `/datos` (servidor: `/mnt/costes`; PC: vacío → `./salidas`)
`CONFIG_CLAVE`	Clave para guardar en `/configuracion` (vacía = sin clave)

> Si `COSTES_SALIDA_XLSX` no está definida, `exportar_costes.py` escribe por defecto en `C:\Users\santiago.arce\Desktop\costes\datos_costes.xlsx`. Defínela en tu `.env`.

5. El cálculo (`desglose.py`)

Las reglas completas están en `docs/reglas-del-coste.html`; aquí va el mapa del código.

5.1 La consulta

`_SQL` es un único CTE (≈ 430 líneas, muy comentado) que devuelve el despiece plano:

1. `componentes`, recursivo: hijos por **fase activa con entradas** (`Fases` + `Fases_Salidas` + `Fases_Entradas`) o, si no hay fase activa, por **conjunto** (`Articulos_Conjuntos`). Cantidades multiplicadas en cascada. Corta ciclos con `Ruta NOT LIKE '%|hijo|%'` y excluye el bucle degenerado conjunto ↔ fase.
2. `precio`: último albarán (`Pedidos_Prov_Lineas`, `Descuento < 100`) → tarifa de proveedor real (`Listas_Precios_Prov_Art`, nunca `IdProveedor = '0'`) → `Trabajos_Subcontratas.CosteUnit`.
3. **Valorable**: hoja con tipo de aprovisionamiento, hoja sin tipo que nunca tuvo fase, o nodo con escandallo que además se compra (trabajo encima: lacado…) salvo que el precio ya esté dentro (freno automático del 2 % o lista `NO_SUMA_PROPIA`).

`NO_SUMA_PROPIA` (5 códigos) se inyecta en el SQL al importar. `IdArticulo` es **VARCHAR**: códigos de 8 dígitos con ceros a la izquierda; trátalos siempre como texto.

5.2 Datos del artículo raíz

`datos_raiz()` hace en **una** conexión las seis consultas pequeñas del artículo buscado: nombre, anulado, `tiempo_operacion` (media ponderada de bonos → mano de obra imputada), `sin_operacion` (casilla `OrdenTrabajo`), `con_orden`, `es_externo` y `coste_propio`.

5.3 Árbol, rollup y avisos

- `construir_arbol(df, codigo, …)` monta el árbol y suma material y operación de abajo arriba (redondeo a **6** decimales; con 4, costes de 10^{-6} desaparecían). Operación = min/pieza × cantidad × coste_hora / 60.
- `avisos_arbol(arbol)` clasifica cada pieza en el primer motivo que cumple: *Sin tiempo de operación* → (hermana de nesting, sin aviso) → *Sin escandallo activo* → *Sin tipo de aprovisionamiento* / *Sin precio de compra*.
- **Completo** = sin avisos de material. *Sin tiempo* no tumba el completo (criterio común a web, lote, pasada y vistas SQL).
- `lineas_coste(arbol)` da el coste partido por pieza (una pieza en varias ramas sale una vez sumada); es lo que se guarda como detalle.

5.4 Caché y coste/hora

- `_recordado()` guarda cada consulta por (tipo, código) **10 minutos** (`CACHE_TTL_S`), con un máximo de 500 artículos. `olvidar_articulo()` la invalida (botón *Recalcular*). `leido_a()` dice de cuándo es el dato que se enseña.
- `coste_hora_operario()` lee `core.cfg_coste_parametro` (clave `coste_hora_operario`), lo recuerda 10 minutos y, si Postgres no responde o no hay fila, usa `COSTE_HORA_DEFECTO = 17`. `/configuracion` llama a `olvidar_coste_hora()` al guardar, pero **solo en el proceso web**: el vigilante lo verá a los 10 minutos como mucho.

6. Publicación (`exportar_costes.py`)

`ejecutar(pg, solo_pendientes, …)`:

1. Inserta una fila «en curso» en `core.log_coste_recalculo`.
2. Lee los ids: `core.cfg_catalogo_coste WHERE activo` (o solo los no completos con `--solo-pendientes`).
3. `calcular()` por artículo → fila de coste, huecos y detalle. Un artículo que falla se publica con `error` y no tumba la pasada.
4. `publicar_calculados()`: añade del ERP, **en bloque**, la familia, el precio de la tarifa de venta vigente y los datos de cada hueco; después `publicar()` sustituye las tablas **en una sola transacción** (`TRUNCATE` + `INSERT`, o `DELETE` de esos ids si es parcial). El detalle y el histórico van en un `SAVEPOINT`: si fallan, el coste se publica igual.
5. Cierra el log y escribe el Excel (`exportar_fichero`). Si el Excel falla (fichero abierto, ruta inexistente) solo avisa.

`recalcular_articulo()` hace lo mismo para un único artículo (botón *Recalcular* del buscador), sin escribir en el log de pasadas.

Objetos de Postgres (esquema `core`)

Objeto	Fichero	Qué es
`cfg_catalogo_coste`	01	Lista de artículos a costear (`activo`, `nota`)
`dim_coste_escandallo`	01 (+05)	Una fila por artículo: costes, contadores de huecos, `completo`, `error`, familia, tarifa
`fact_coste_hueco`	01	Qué pieza bloquea a qué artículo y por qué
`log_coste_recalculo`	01	Traza de pasadas (`manual` / `programado`)
`v_coste_hoja10`	02	**Contrato con el libro de Finanzas**: 4 columnas exactas, código como entero, solo material. No cambiar sin tocar el `VLOOKUP` del libro
`v_coste_pendiente`, `v_coste_resumen`	03	Piezas pendientes y KPIs de `/costes`
`v_coste_articulo`	04	Hoja «Articulos» del libro
`v_rentabilidad`	05	Coste frente a tarifa. Un coste incompleto **nunca** recibe tramo de margen
`cfg_coste_parametro`, `log_coste_parametro`	06	Parámetros editables y su historial (con IP)
`fact_coste_linea`, `hist_coste_articulo`, `hist_coste_linea`	07	Detalle vigente por pieza e histórico (las líneas solo cuando cambia la huella md5)
`cfg_coste_vigilancia`	`vigilar_costes.py`	Fecha de la última pasada programada

Todos los ficheros son idempotentes. `py exportar_costes.py --crear` los aplica todos en orden y en una transacción (necesario en un despliegue limpio). `06` y `07` se aplican además solos al guardar la configuración y al publicar.

`/evolucion` y `/piezas` leen además `core.fact_ventas` / `fact_ventas_lineas` (ventas de 12 meses), que mantiene el ETL de `gyc_analytics`, no esta aplicación.

7. Web

- Todas las respuestas del buscador salen de `/api/desglose`, que devuelve el árbol completo como JSON; la plantilla `index.html` lo pinta (etiquetas, avisos, colores).
- **Lote**: `POST /lote` lee la primera hoja *visible* del Excel, busca la columna de códigos (`articul`, `codig`, `id`, `ref`), rellena ceros hasta 8 dígitos y lanza un hilo. Los artículos del catálogo se toman de lo publicado (columna *Calculado* con la fecha) y el resto se calcula contra el ERP. Estado en memoria (`_lotes`), descargable 1 hora; máximo 2.000.
- `/costes`: un solo recálculo a la vez **por proceso** (`_estado` + lock).
- `/configuracion`: para añadir un parámetro basta una entrada en `PARAMETROS` (`web_config.py`) y leerlo donde se use; página, guardado e historial son genéricos.

8. Tests

```
pytest tests/ -v                                    # en local, con .env y red al ERP
docker exec -w /app costemp python -m pytest tests/ -v  # dentro del contenedor
```

Archivo	Qué protege	Necesita ERP
`test_regresion_costes.py`	Bugs reales: precio propio duplicado, ciclo fase ↔ conjunto, 100 % descuento, trabajo externo sin aviso de tiempo, media de bonos sobre teórico, material + operación = total	**Sí**
`test_menos_erp.py`	Cuándo toca la pasada, caché (una consulta por artículo, caducidad, límite), botón Recalcular	No (ERP falso)
`test_coste_hora_y_lote.py`	Coste/hora desde Postgres o 17 por defecto, lote con catálogo publicado, `/configuracion`	No
`test_detalle_y_analisis.py`	El detalle suma lo mismo que el árbol, huella, simulador de `/piezas`	No

Los tests de regresión son **de integración contra el ERP real**: si alguien cambia datos en el ERP (una fase, un precio), un test puede fallar sin que el código esté mal (pasó con el 61000165, commit `77e8bc9`). Antes de cambiar el valor esperado, comprueba que se sigue cumpliendo la **relación** que el test describe.

Comprobación de extremo a extremo tras tocar el cálculo:

```
py validar_costes.py --todos     # lo publicado == lo que calcula la app ahora (tolerancia 0,0001 €)
```

Gates

- `pre-push` **local** (`git config core.hooksPath scripts/git-hooks`): lanza un contenedor efímero desde `costemp:latest`, copia el código y pasa los tests. Necesita Docker, la imagen construida y el `.env`.

- **CI** (`.github/workflows/tests.yml`): en cada push y PR, en el runner self-hosted del servidor (los runners de GitHub no llegan a `10.0.0.6`). Usa el `.env` de `/home/goyco/apps/costes-articulo/.env` y la imagen `costemp:latest`.

Ambos **copian archivo por archivo** el código dentro del contenedor. Si creas un módulo Python nuevo, añádelo a las dos listas de `docker cp`, o todos los tests fallarán con un `ModuleNotFoundError`. Y como reutilizan la imagen ya construida, **una dependencia nueva en `requirements.txt` no está** hasta reconstruir `costemp:latest` (`docker compose build`).

9. Despliegue

Servidor `10.0.0.12`. Detalle de la preparación (montaje CIFS, credenciales, comprobaciones) en `despliegue.md`.

```
cd /home/goyco/apps/costes-articulo      # carpeta del clon en el servidor (la del .env del CI)
git pull
docker compose up -d --build
docker compose exec costemp python exportar_costes.py --crear --solo-pendientes   # si hay SQL nuevo
docker compose ps
```

No hay despliegue automático: el CI solo pasa tests. Tras un `--build`, la imagen nueva también la usarán el CI y el `pre-push`.

Contenedor	Comando	Notas
`costemp`	`waitress-serve --port=5000 app:app`	Publicado en el host en **5010**
`costemp-vigilante`	`python vigilar_costes.py --bucle 900 --cada-dias 3 --a 05:00`	Mira Postgres cada 15 min; solo toca el ERP en la pasada

Ambos con `TZ=Europe/Madrid` (sin ello las fechas quedaban 2 h por detrás y la pasada saltaba con la planta trabajando) y con el volumen `${COSTES_DATOS_DIR:-./salidas}:/datos`.

La hora de la pasada (05:00) está elegida lejos de las copias del ERP (01:00 y 06:30).

Tras el primer despliegue en el servidor había que **borrar las tareas programadas del PC de Santiago** («GYC Costes vigilancia» y «GYC Costes catalogo»), o habría dos procesos publicando lo mismo. Comprueba que ya no existen (`schtasks /query | findstr Costes`).

10. Recetas

Añadir o quitar artículos del catálogo

```
INSERT INTO core.cfg_catalogo_coste (idarticulo, nota) VALUES ('12101021', 'motivo');
UPDATE core.cfg_catalogo_coste SET activo = false WHERE idarticulo = '12101021';   -- quitar sin
perder el registro
```

Entran en la siguiente pasada (o en `/costes` → *Recalcular todo*). Los códigos del catálogo deben ser numéricos y sin ceros a la izquierda para entrar en `v_coste_hoja10` (el libro los cruza como número).

Cambiar una regla del cálculo 1. Lee la sección correspondiente de `reglas-del-coste.html` y el comentario del CTE. 2. Cambia `desglose.py` y añade un test de regresión con el artículo real que lo motiva. 3. `pytest tests/ -v` y, tras desplegar y recalcular, `py validar_costes.py --todos`. 4. Actualiza `reglas-del-coste.html` (es documentación de referencia para Finanzas).

Añadir un dato a lo publicado: columna con `ADD COLUMN IF NOT EXISTS` en un `sql/0N_*.sql` nuevo o existente, rellénala en `calcular()` o en `publicar_calculados()` (así llega a la pasada y al recálculo individual) y añádela al `INSERT` de `publicar()`.

Añadir un parámetro editable: entrada en `web_config.PARAMETROS` y lectura en el sitio que lo use (siguiendo el patrón de `coste_hora_operario()`, con valor por defecto si Postgres falla).

Añadir un módulo `.py`: impórtalo donde toque y añádelo a los `docker cp` del workflow y del hook `pre-push`.

11. Trampas conocidas y deuda técnica

Tema	Detalle
Trabajo sin subir	Ver la nota del principio: `main` no tiene aún configuración, histórico ni análisis
Rutas del PC de Santiago en el código	`exportar_costes.SALIDA` (si falta `COSTES_SALIDA_XLSX`) y `CATALOGO_TXT` (respaldo si el catálogo está vacío) apuntan a su escritorio
Recálculos simultáneos	El bloqueo de `/costes` es por proceso: una pasada del vigilante y un *Recalcular todo* manual pueden coincidir. La publicación es transaccional (gana la última), pero cargan el ERP el doble
Caché por proceso	La web y el vigilante no comparten caché ni el coste/hora recordado (hasta 10 min de desfase)
Lotes en memoria	Se pierden si se reinicia el contenedor
Tests contra el ERP real	Pueden fallar por cambios de datos, no de código
`docker cp` en CI y hook	Lista manual de archivos; olvidar uno rompe todos los tests
`v_coste_hoja10`	Contrato rígido con el libro de análisis (columnas, orden, tipo entero, solo material)
Reparto materia prima / mercadería	Depende de `IdTipoAprovisionamiento`, un campo manual y poco relleno en el ERP
`core.dim_coste_articulo`	Tabla antigua y muerta (última carga 2022). No usarla ni borrarla
Sin autenticación	Cualquiera en la red puede lanzar recálculos; `/configuracion` solo protegida si hay `CONFIG_CLAVE`
`.gitignore` ignora `*.pdf`	Excepto la carpeta `documentación/`, que sí se versiona

12. Checklist de relevo

- ☐ Acceso al repositorio `GYCai-tech/CosteMP` y a su runner self-hosted.
- ☐ Acceso SSH a `10.0.0.12` con permisos de Docker; confirmar la carpeta del clon.

- ☐ Credenciales de solo lectura del ERP y de `gyc_etl` en `gyc_analytics` (y el `.env` del servidor a buen recaudo).
- ☐ Subir y desplegar el trabajo en curso (o descartarlo) para que `main` y producción coincidan.
- ☐ Confirmar que ya no existen las tareas programadas en el PC de Santiago.
- ☐ Definir `COSTES_SALIDA_XLSX` en todos los `.env` y retirar las rutas de su escritorio del código.
- ☐ Saber quién en Finanzas mantiene el libro de análisis y su `VLOOKUP` contra `v_coste_hoja10`.
- ☐ Pasar los tests y `validar_costes.py` una vez antes de hacer cambios.